

Artefactos técnicos y documentación automatizada

Gestión de artefactos

1. introducción

En el desarrollo de software, no solo se produce código, sino también una gran cantidad de archivos y recursos necesarios para construir, entender y mantener un sistema.

Estos elementos reciben el nombre de **artefactos de software**.

Una correcta gestión de estos artefactos es fundamental para garantizar la **mantenibilidad, escalabilidad y calidad del sistema**.

2. ¿Qué son los artefactos de software?

Un artefacto de software es cualquier producto tangible o intangible creado durante el desarrollo de una aplicación, como código fuente, documentación, diagramas, modelos de datos, scripts de configuración o archivos ejecutables. Estos elementos permiten planificar, diseñar, construir, probar y mantener el software, funcionando como registros del progreso y componentes del producto final.

Los artefactos se pueden clasificar según su propósito en el ciclo de vida:

- Documentación de requisitos: Archivos que definen las necesidades del usuario, como casos de uso o especificaciones funcionales.
- Diseño y arquitectura: Diagramas de flujo, modelos de datos, diagramas de clases y prototipos que guían la construcción.
- Código y ejecutables: El código fuente, archivos de configuración, scripts, bibliotecas y el programa ejecutable final.

3. ¿Por qué son importantes?

Los artefactos son imprescindibles para:

Mantenimiento: Facilitan la resolución de problemas y actualizaciones futuras al servir como material de referencia.

Registro Histórico: Sin ellos, no habría evidencia del trabajo realizado ni de las decisiones tomadas durante el proyecto.

Comunicación: Ayudan a alinear al equipo de desarrollo con las partes interesadas (clientes o usuarios).

4. Gestión de artefactos

La gestión de artefactos consiste en organizar, versionar y mantener todos los elementos de un proyecto de software de manera controlada, con el objetivo de garantizar su correcta evolución a lo largo del tiempo.

No se trata solo de guardar archivos, sino de aplicar criterios, normas y herramientas que permitan que cualquier integrante del equipo pueda:

- Entender el proyecto.
- Modificarlo sin generar errores ni romper componentes relacionados.
- Mantener coherencia entre sus componentes.

En entornos profesionales, una mala gestión de artefactos puede provocar fallos críticos, pérdida de información o retrasos en el desarrollo.

4.1 Organización en carpetas

La estructura del proyecto debe ser clara, lógica y predecible, de modo que cualquier desarrollador pueda ubicar rápidamente los archivos necesarios.

Una estructura estándar de proyecto podría verse así:

Carpeta / Archivo	Descripción
src/	Código fuente del sistema
docs/	Documentación técnica
tests/	Pruebas del sistema
config/	Archivos de configuración
assets/	Recursos (imágenes, archivos)

README.md	Información general del proyecto
-----------	----------------------------------

4.2 Control de versiones

El control de versiones permite registrar y gestionar los cambios realizados sobre los artefactos a lo largo del tiempo.

La herramienta más utilizada es Git ya que nos permite:

- Guardar historial de cambios.
- Recuperar versiones anteriores.
- Trabajar en paralelo con otros desarrolladores.
- Evitar sobrescribir trabajo ajeno.

El control de versiones es clave para la **colaboración** y **trazabilidad** del proyecto.

4.3 Documentación estructurada

La documentación debe estar organizada, actualizada y accesible. No alcanza con tener documentación: debe ser útil y mantenible.

Tipos de documentación:

- README: descripción general del proyecto.
- Documentación técnica: arquitectura, decisiones de diseño.
- Guías de uso: cómo utilizar el sistema.
- Comentarios en código.

Las "4 C" de la buena documentación: **Clara** (fácil de entender), **Concisa** (sin redundancias innecesarias), **Correcta** (exactamente precisa) y **Completa** (con toda la información que el lector objetivo necesita).

4.4 Estándares de mantenibilidad

La mantenibilidad es una de las características más importantes de la calidad del software.

Se refiere a la capacidad de un sistema para ser modificado, corregido o mejorado de forma sencilla, rápida y segura, sin introducir nuevos errores.

Un sistema mantenible permite:

- Adaptarse a nuevos requerimientos.
- Corregir errores fácilmente.

- Ser comprendido por nuevos desarrolladores.
- Evolucionar sin necesidad de rehacerlo.

5. Herramientas de gestión de artefactos

Existen herramientas específicas para cada aspecto de la gestión de artefactos. Conocerlas permite elegir la más adecuada según el tipo de proyecto y el tamaño del equipo:

Herramienta	Categoría	Función principal	Uso típico
Git	Control de versiones	Versionado de código y documentación	Todo tipo de proyectos
GitHub / GitLab	Plataforma Git + colaboración	Repositorio remoto, pull requests, CI/CD	Equipos de desarrollo
SonarQube	Análisis de código	Métricas de mantenibilidad, deuda técnica	Proyectos empresariales
Confluence	Wiki colaborativa	Documentación técnica centralizada	Equipos con Jira
Notion	Gestión del conocimiento	Wikis, backlog, documentación liviana	Equipos pequeños/medianos
Doxygen	Generación de docs	Genera docs desde comentarios en el código	Proyectos C++, Java, Python
Document360	Base de conocimiento	Documentación con análisis de uso	Productos con usuarios externos
Jira / Trello	Gestión de tareas	Backlog, sprints, seguimiento de tareas	Metodologías ágiles